Lecture 11
Modular Programming

UESTC 2004: Embedded Processors

Dr Luiz Felipe Aguinsky

Most slides were prepared by Dr Mark D. Butala, Dr Yao Sun, Dr Lina Mohjazi, Dr Ahmad Taha, and Dr Syed M. Raza

Housekeeping

- Office Hours – Wednesday 2-3:30pm and Thursday 10:00 at Room 421
  - Next week is my last! Office Hours will be 1st and 2nd June

- Lab 4 involves interfacing with the TMP102 temperature sensor
  - Involves I^2C serial communication
  - See Lecture 10
  - Today: brief re-cap
  - Material already available on Moodle

Course Project – Teamwork!

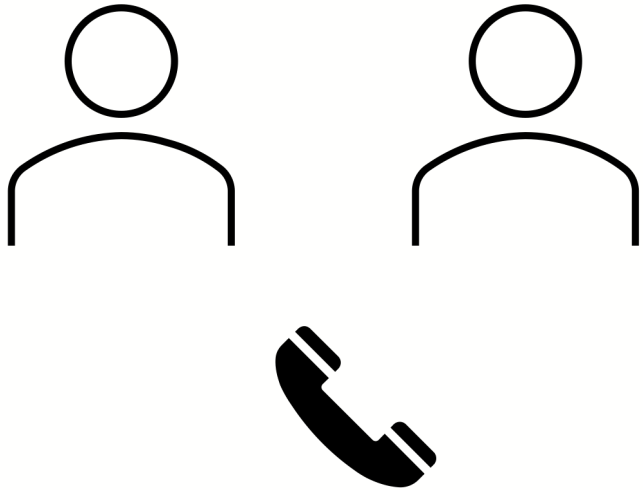
Main (Written) Assignment – Cold-Chain Transport Condition Logger

Embedded Processors UESTC & HN 2004 School of Engineering, University of Glasgow

The Assignment:

The aim of this project is to **design, implement, and test** an embedded system that operates as a **Cold-Chain Transport Condition Logger**, using input/output peripherals connected to the **NUCLEO-L432KC** board.

You will work in **groups of two (Working individually or in a group of 3 is acceptable only if approved, in writing, by the Course Coordinator)**. By now you should have already submitted your group members on Moodle. The project must be developed collaboratively, with **clear task allocation and individual responsibility**, and evidence of how each group member contributed to the system's development.



Course Project – Teamwork!

Main (Written) Assignment – Cold-Chain Transport Condition Logger

Embedded Processors UESTC & HN 2004 School of Engineering, University of Glasgow

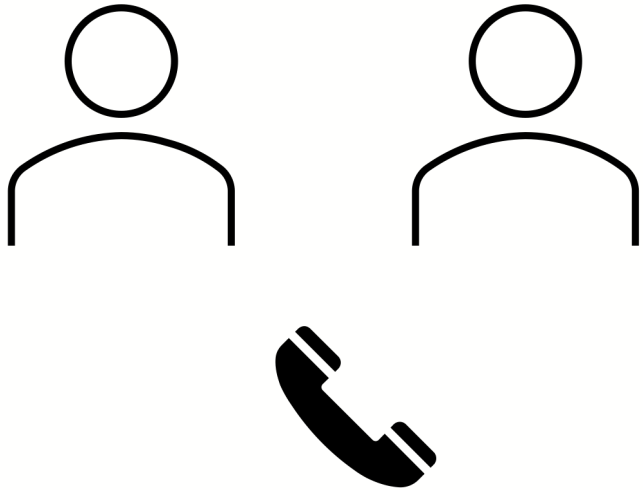
Group Work Requirements & General Guidance:

This project is assessed as a **group** design and development activity. However, you must:

- allocate **specific responsibilities** to each group member (e.g. sensor interfacing, power management, system logic, testing, documentation).
- submit a task allocation and contribution statement as part of your report, clearly detailing:
    - who was responsible for what,
    - how work was coordinated,
    - and how integration was managed.

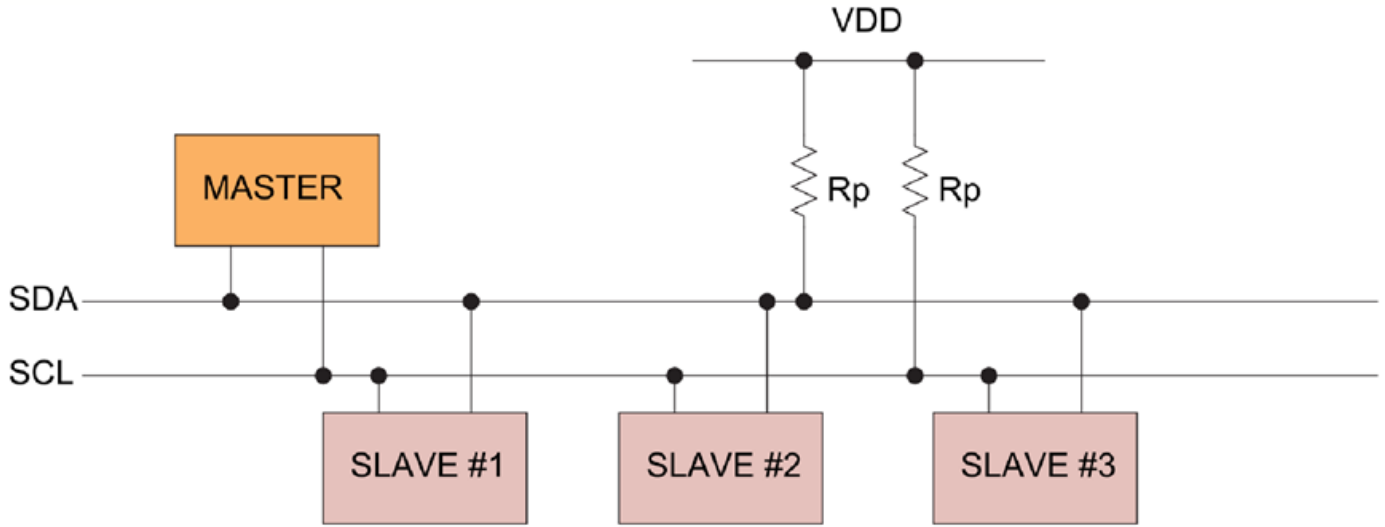
  Failure to clearly demonstrate individual contributions may affect your overall mark.

2. A 3-5 minutes video demonstration of your system while in operation. (Please explain the operation with your voice during recording and make sure to do rigorous testing to showcase your system's full operation – all group members are to take part in this)



I²C: A Simple, Synchronous, Serial Communications Protocol

- Inter Integrated Circuit (I²C)
  - Serial Data Line (SDA): Bi-directional (one direction at a time – half duplex) - transmit data, address, and control signals
  - Serial Clock Line (SCL): for synchronisation



I²C: Mbed Bidirectional Communication Example

```c
#include "mbed.h"

#define BLINKING_RATE 500ms

#define I2C_FREQUENCY 400 * 1000  // 400 kHz
#define I2C_SLAVE_ADDRESS 0x12

#define BUFFER_LENGTH 20

int main() {
    DigitalOut led(LED1);

    char buf[BUFFER_LENGTH];
    char counter = 0;

    I2C i2c_master(D4, D5);  // SDA, SCL
    i2c_master.frequency(I2C_FREQUENCY);
```

Controller

```c
    while (true) {
        led = !led;
        ThisThread::sleep_for(BLINKING_RATE);

        int write_return = i2c_master.write(I2C_SLAVE_ADDRESS, &counter, 1);
        if (write_return != 0) {
            printf("Master write received NACK!\n");
        }

        int read_return = i2c_master.read(I2C_SLAVE_ADDRESS, buf, BUFFER_LENGTH);
        if (read_return != 0) {
            printf("Master read received NACK!\n");
        }
        printf("%s\n", buf);
        buf[0] = '\0';

        counter++;
    }
}
```

I²C: Mbed Bidirectional Communication Example

```
#include "mbed.h"

#define DELAY 30ms

#define I2C_SLAVE_ADDRESS 0x12
#define I2C_FREQUENCY 400 * 1000  // 400 kHz

#define MESSAGE_LENGTH 20

int main() {
    I2CSlave slave(D4, D5);
    // CALL FREQUENCY METHOD BEFORE ADDRESS METHOD!!!
    slave.frequency(I2C_FREQUENCY);
    slave.address(I2C_SLAVE_ADDRESS);

    DigitalOut led(LED1);
    led = 0;

    char buf = 0;
    char msg[MESSAGE_LENGTH];
```

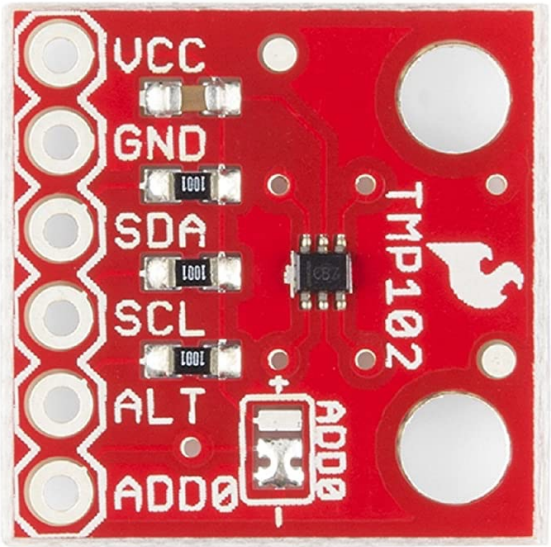
```
    while (1) {
        led = !led;
        ThisThread::sleep_for(DELAY);

        int i = slave.receive();
        int return_value;
        switch (i) {
            case I2CSlave::ReadAddressed:
                printf("ReadAddressed\n");
                // 19 characters + \0 termination = 20
                sprintf(msg, "Slave receieved %3d", buf);
                return_value = slave.write(msg, MESSAGE_LENGTH);
                if (return_value != 0) {
                    printf("Slave write error\n");
                }
                break;
            case I2CSlave::WriteGeneral:
                assert(0);
                break;
            case I2CSlave::WriteAddressed:
                return_value = slave.read(&buf, 1);
                if (return_value != 0) {
                    printf("Slave received 0 bytes\n");
                }
                printf("WriteAddressed: %d\n", buf);
                break;
            case I2CSlave::NoData:
                printf(".");
                break;
            default:
                assert(0);
        }
    }
}
```

Worker

- `DELAY 30ms` was determined by experimentation
- Note the comment about `frequency`

TMP102 Temperature Sensor

- Interface the TMP102 sensor to the NUCLEO L432KC mbed board using I^2C
- We will need to configure the I^2C on our mbed board in accordance with the temperature sensor requirements
- When in doubt, refer to the datasheet (it has been posted under Lecture 10 on Moodle)



I²C Protocol Implementation – TMP102 (Continuous Reading)

```c
#include "mbed.h"

#define I2C_MASTER_SDA PA_10
#define I2C_MASTER_SCL PA_9

#define TMP102_ADDRESS 0x90

#define CONVERSION 0.0625

#define DELAY_US 500 * 1000  // i.e., 500ms

int main() {
    I2C tmp102(I2C_MASTER_SDA, I2C_MASTER_SCL);
    tmp102.frequency(400 * 1000);

    char tmp102_config[3] = {0x01, 0x60, 0xA0};
    tmp102.write(TMP102_ADDRESS, tmp102_config, 3);

    tmp102_config[0] = 0x00;
    tmp102.write(TMP102_ADDRESS, tmp102_config, 1);

    char temperature[2];
    while (1) {
        tmp102.read(TMP102_ADDRESS, temperature, 2);
        unsigned short M = temperature[0] << 4;
        char L = temperature[1] >> 4;
        M += L;
        printf("Temperature register value = %u (byte1=0x%X byte2=0x%X)\n", M, temperature[0], temperature[1]);
        printf("Temperatre in Celsius = %.3f\n", M * CONVERSION);
        wait_us(DELAY_US);
    }
}
```

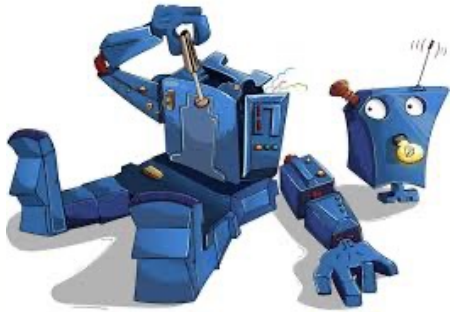
Lesson Plan

Aim	• Understand and apply modular programming
Learning Objectives	By the end of this lesson you will: • Understand the principle of modular programming • Write Functions • Apply modular programming
Assumed Previous Knowledge	• C Programming

Click for Course Specifications & Learning Outcomes

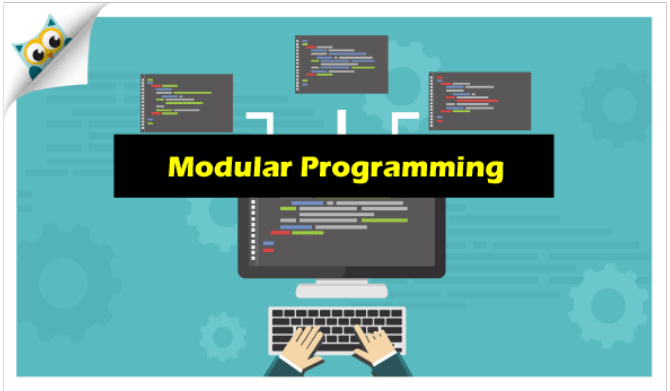
Why Modular Programming?

- Numerous challenges when tackling an embedded system design project
- Our target is to have a design that fulfils the following:
  - The code is readable, structured and documented
  - The code can be fully and extensively tested for performance
  - The development reuses existing code to keep development time short – Time to market is reduced £
  - The code design supports multiple engineers working on a single project simultaneously
  - Future upgrades to code can be implemented efficiently



Why Modular Programming?

- It is wise first to consider the software design structure, particularly with large projects that perform multiple dependent or independent functions.
- It is not possible to program all functionality into a single control loop.
- Breaking up code into understandable features should always be a design consideration.



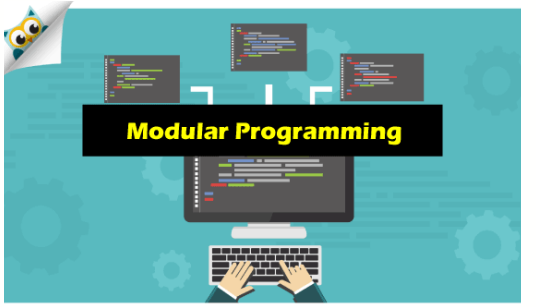
MODULAR PROGRAMMING!

- Without modularity in embedded systems:
  - Hard to debug hardware/software interactions
  - Changes propagate unpredictably
  - Testing becomes system-level only (poor practice)
- Modularity enables design → implementation → testing → integration cycle!

Procedural vs Modular Programming?

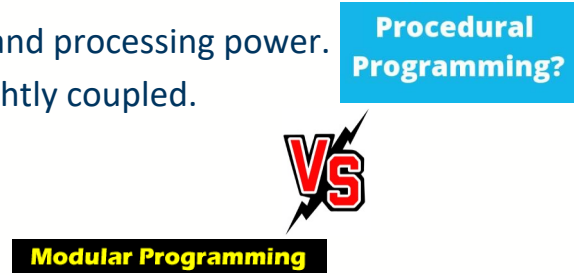
- **Procedural programming:**
  - An imperative programming paradigm where the program is structured around procedures or routines, which are sets of instructions executed sequentially.
  - It emphasises a step-by-step approach to solving problems, with functions or procedures manipulating shared data.

- **Modular programming:**
  - A programming paradigm that emphasises breaking down a program into smaller, self-contained modules, each responsible for a specific task or functionality. These modules can be developed and tested independently.
  - It's like assembling a puzzle: you break the problem into smaller pieces (modules), solve each piece separately, and then combine them to achieve the desired result.
  - In embedded systems, modules can correspond to hardware peripherals (e.g., LED, Button, TemperatureSensor) or software blocks (e.g., bootloaders, state machine, data logger, algorithms) or logical layers (e.g., Hardware Abstraction Layer (HAL): Drivers interacting directly with registers).



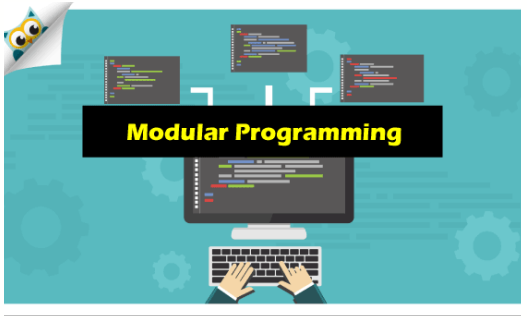
Procedural vs Modular Programming?

- **Procedural programming**
  - Sequential Flow: It follows a linear sequence of instructions.
  - Resource Efficiency: It typically consumes fewer system resources like memory and processing power.
  - Limited Reusability: Code reusability might be limited as functions tend to be tightly coupled.
  - Difficult to Manage with Complex Programmes.
  - Testability: Difficult to test without full hardware
- **Modular programming**
  - Encapsulation: It promotes breaking down the code into smaller, manageable modules. Each module encapsulates a specific functionality, making the code easier to understand, maintain, and reuse.
  - Code Reusability: It fosters code reusability since modules can be easily integrated into different projects or parts of the same project.
  - Scalability: It is easy for developers to add, remove, or modify modules without affecting other parts of the system.
  - Testability: Each module can be tested in isolation using stubs (a dummy piece of code used to replace a lower-level module that is not yet written or ready).
  - Increased Overhead: It introduces some overhead due to function calls and module interactions, impacting performance in systems with strict resource constraints. **(acceptable on modern MCUs; prioritise maintainability)**



Key Terminologies Used with Modular Programming

- **Modules:**
  - The backbone of a modular program. Each module encapsulates a specific functionality or a set of functionalities.
  - They should have well-defined boundaries, interfaces, and responsibilities. They should be designed to be reusable and easily replaceable.

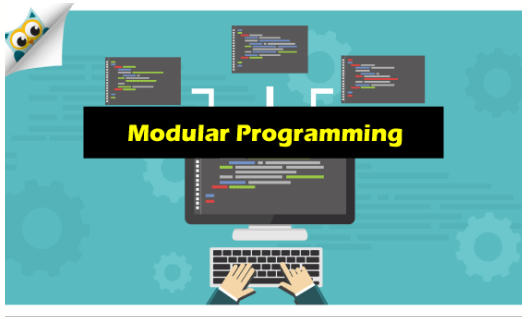


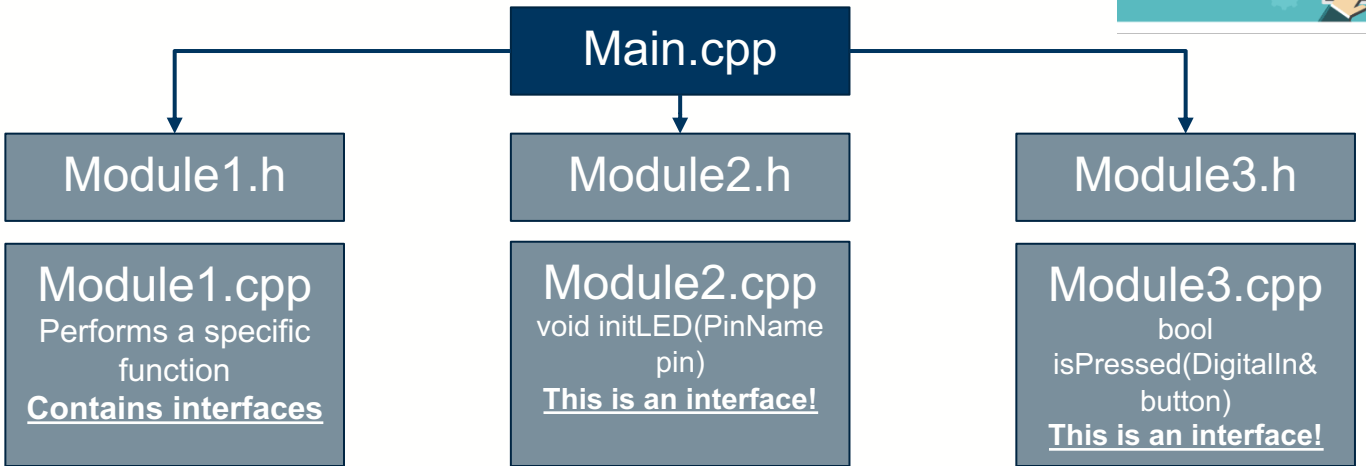
Main.cpp

Module1.h	Module2.h	Module3.h
Module1.cpp **Performs a specific function**	Module2.cpp **Control an LED**	Module3.cpp **Read input from a push button**

Key Terminologies Used with Modular Programming

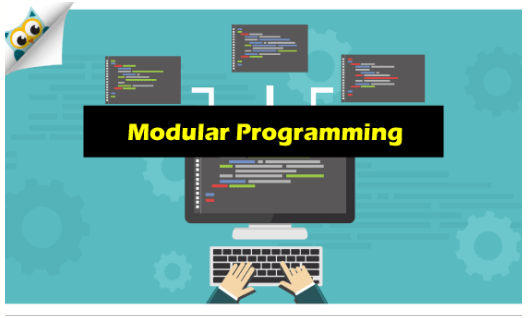
- **Interfaces:**
  - They define how modules interact with each other. They specify the methods, functions, or protocols that modules can use to communicate.
  - They provide a clear contract between modules, detailing the inputs, outputs, and behaviours expected from each module.
  - A well-designed interface is stable, minimal, and complete.





Key Terminologies Used with Modular Programming

- **Interfaces:**
  - They define how modules interact with each other. They specify the methods, functions, or protocols that modules can use to communicate.
  - They provide a clear contract between modules, detailing the inputs, outputs, and behaviours expected from each module.
  - A well-designed interface is stable, minimal, and complete.



❌ Bad Design (Exposes Hardware Internals)

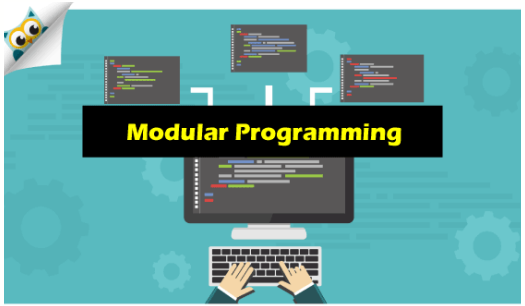
```
// Fragile: Changes to hardware timers or registers break the app
void LED_InitWithTimer(PinName pin, uint8_t timerChannel, uint32_t frequency);
```

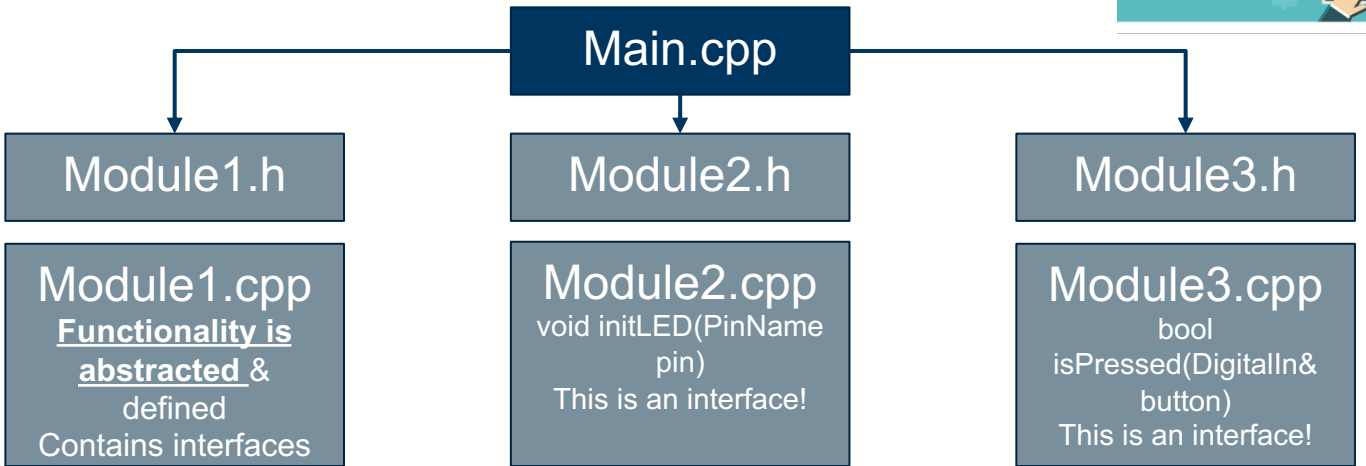
✅ Good Design (Stable, Minimal, Complete)

```
// Interface (led.h) - Hardware is completely hidden
void LED_Init(PinName pin);
void LED_On(PinName pin);
void LED_Off(PinName pin);
void LED_Toggle(PinName pin);
```

Key Terminologies Used with Modular Programming

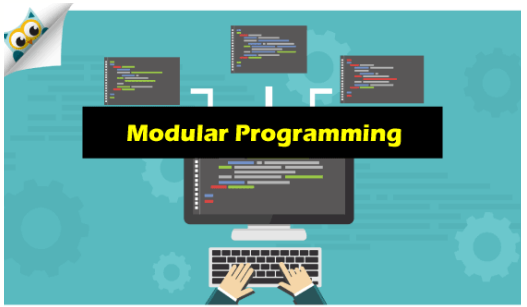
- **Abstraction:**
  - Hiding the implementation details of a module's functionality, exposing only the essential features through its interface.
  - It allows modules to be used without needing to understand how they are implemented internally, promoting code reuse and simplifying system design.
  - It allows you to change, e.g., an LED from active-high to active-low without modifying the code that calls *"LED_on()"*.

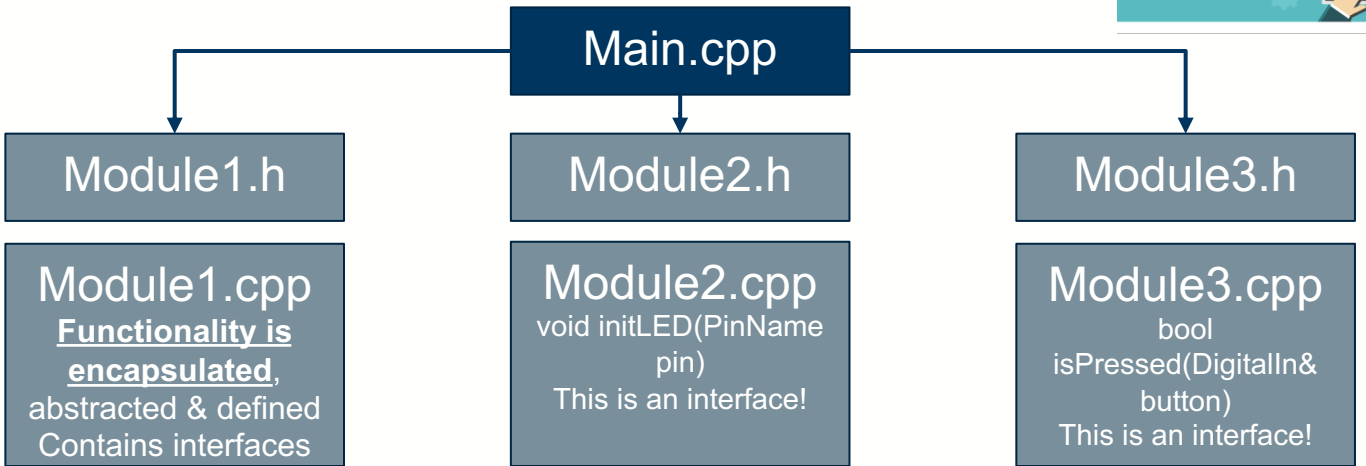




Key Terminologies Used with Modular Programming

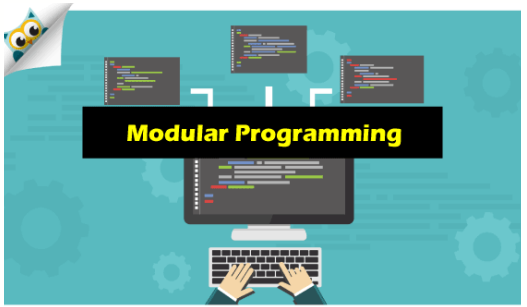
- **Encapsulation:**
  - Encapsulation bundles data and the methods or functions that operate on that data into a single unit, i.e., a module.
  - Encapsulation protects a module's internal state from external manipulation, allowing controlled access to its functionality through well-defined interfaces.
  - **Avoid direct access to module variables!**





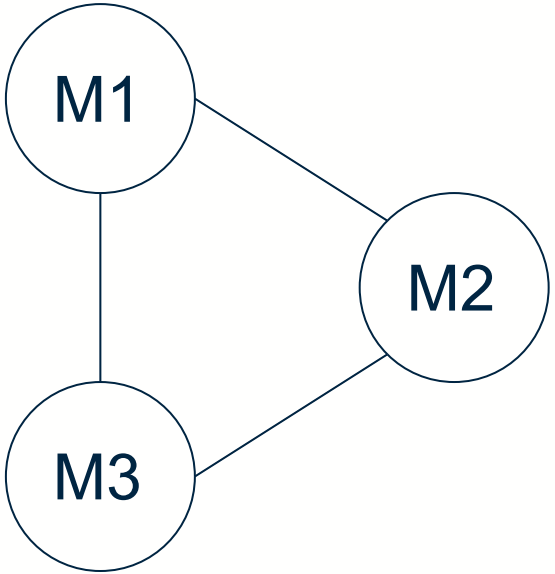
Key Terminologies Used with Modular Programming

- **Dependencies:**
  - It defines the relationships between modules. A module may depend on the functionality provided by other modules to perform its tasks.
  - Managing dependencies is crucial in modular programming to ensure that changes to one module do not unintentionally affect other modules.
  - Avoid circular dependencies (A includes B, B includes A). Use forward declarations or restructure.



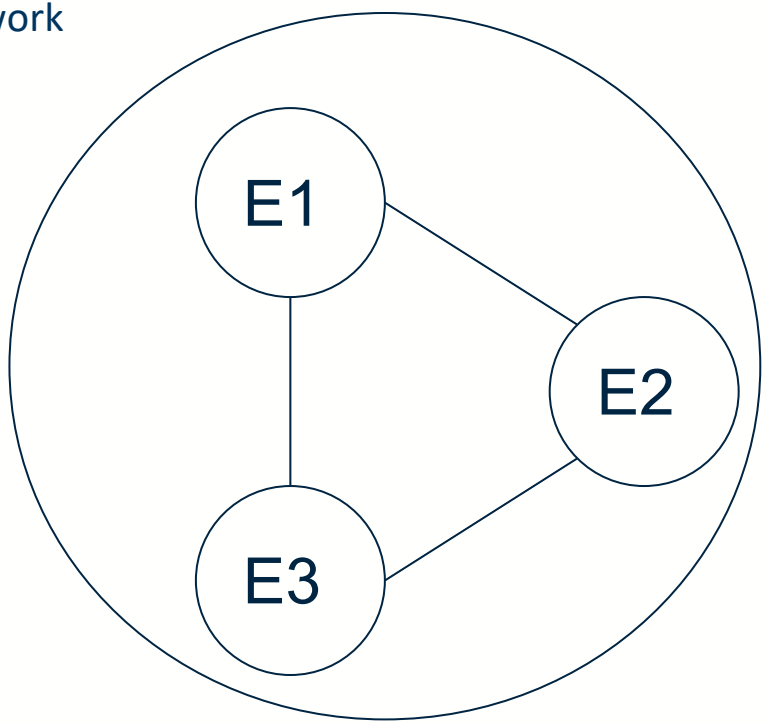
Coupling and Cohesion in Embedded Systems

- **Coupling:** The degree of interdependence between software modules.
- High coupling means that modules are closely connected and changes in one module may affect other modules.
- Low coupling means that modules are independent, and changes in one module have little-to-no impact on other modules.
- Logical coupling refers to relationships between program abstractions.
- Physical coupling refers to relationships between files, such as source/header file inclusions or library linkage.



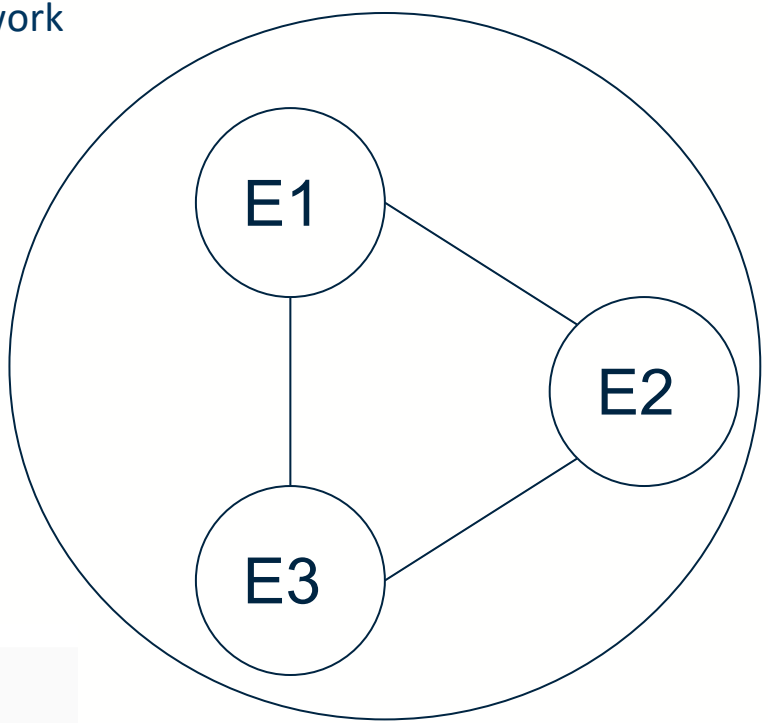
Coupling and Cohesion in Embedded Systems

- **Cohesion:** The degree to which elements within a module work together to fulfill a single, well-defined purpose.
- High cohesion means that elements are closely related and focused on a single purpose.
- Low cohesion means that elements are loosely related and serve multiple purposes.



Coupling and Cohesion in Embedded Systems

- **Cohesion:** The degree to which elements within a module work together to fulfill a single, well-defined purpose.
- High cohesion means that elements are closely related and focused on a single purpose.
- Low cohesion means that elements are loosely related and serve multiple purposes.



- **Low Coupling & High Cohesion**
- **Bad:**

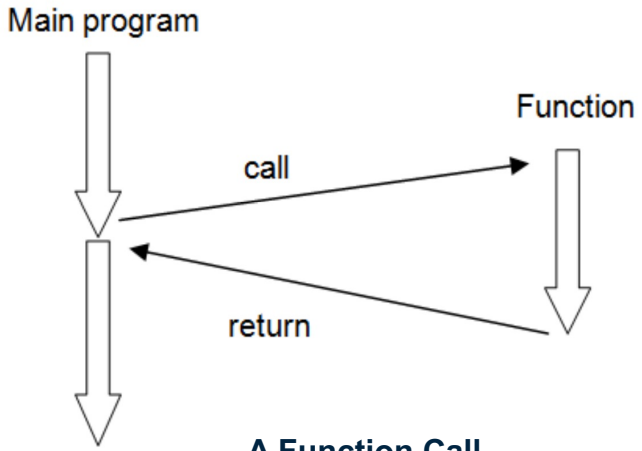
```
led_state = button.read();
```

- **Good:**

```
if (Button_isPressed()) { LED_toggle(); }
```

Modular Programming: Functions and Subroutines

- A function (sometimes called a subroutine) is a portion of code within a larger program.
- The function performs a specific task and is relatively independent of the main code.
- Functions can be used to manipulate data; this is particularly useful if several similar data manipulations are required in the program.
- Data values can be input to the function and the function can return the result to the main program.
- Functions can be used with no input or output data, simply to reduce code size and to improve readability of code.
- A crucial building block of modular programming.



A Function Call

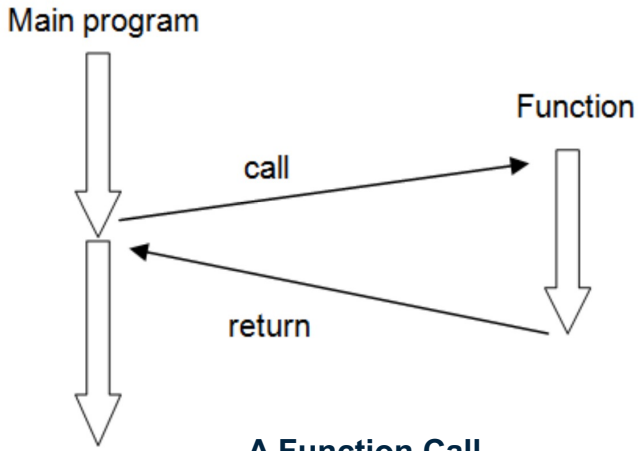
Modular Programming: Benefits and Drawbacks of Functions

Benefits:

- A function is written once and compiled into one area of memory, irrespective of the number of times that it is called.
- Functions allow clean and manageable code to be designed, allowing software to be well structured and readable.
- Functions also enables the practice of modular coding.

Drawbacks:

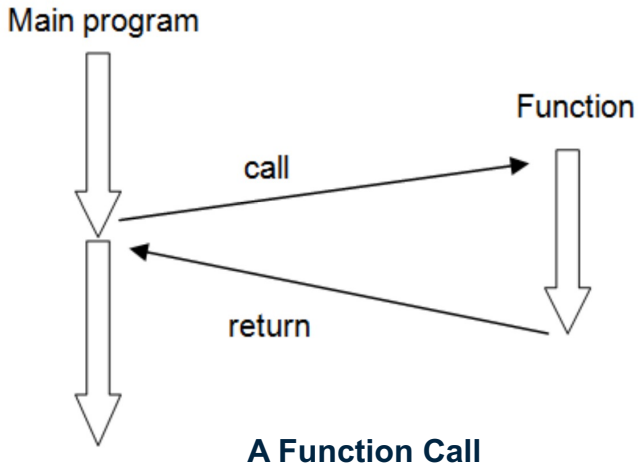
- There is a small execution time overhead in storing program position data and jumping and returning from the function.
- Nesting functions can make the software complicated and challenging to follow.
- C functions can only return a single value. Arrays of data cannot be passed to or from a function (***there are work arounds of course!***).



A Function Call

Modular Programming: Scope Rules of Functions

- The scope rules of a language are the rules that govern how an object may be accessed by various parts of your program.
- It refers to where an identifier is accessible in the program.
- The scope rules determine what code has access to a variable.
- It determines how long it is available to your program and where it can be accessed.
- It determines the lifetime of a variable.
- There are three types of variables:
  - Local variables,
  - Formal parameters, and
  - Global variables.



A Function Call

Modular Programming: Scope Rules of Functions

Local Variables

- Declared inside a function.
- Used, ONLY, by statements located within the function.
- Exist only while the block of code in which they are declared is executing.

Example:

```
for(int i=0; i < 5; i++) {
   printf("i = %d\n", i);
}
i += 10;  // compilation
          // error
```

Formal Parameters

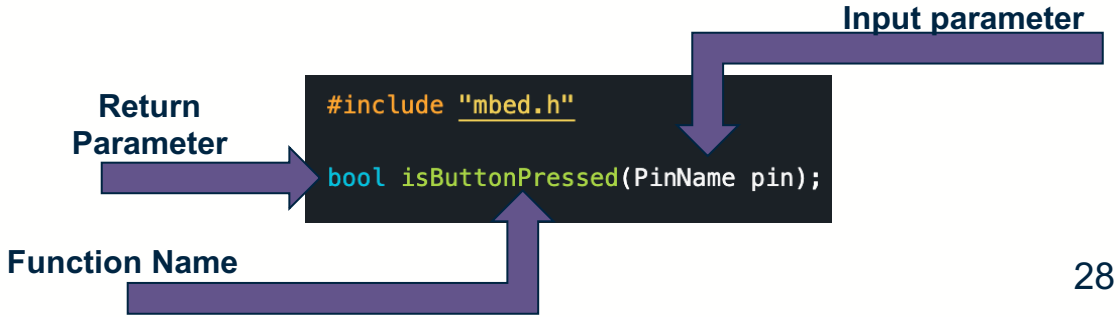
- If a function uses arguments, then variables are needed to accept the values of those arguments.
- These are called the formal parameters (input or output).
- They behave like local variables.
- They must match the type of the arguments you will pass to the function.

Global Variables

- The opposite of local variables.
- Their scope extends to the entire program.
- Can be used by any piece of code, and maintain their values during the entire execution.
- Global variables are helpful when the same data is used by several functions.

Modular Programming: Structure of Functions

- A function consists of two parts, function declaration/prototype and function definition.
- Function declaration provides information about the function's name, return type, and parameters (if applicable).
- It acts as a contract between the function and the rest of the program, allowing other parts of the program to use the function without knowing its implementation details. Functions are declared in ".h" files.
- The declaration ends with a semicolon (;), indicating that it's a declaration rather than a definition.
- A declaration is the interface contract. It tells other modules what the function does without revealing how



Modular Programming: Structure of Functions

- Function definition contains the actual implementation of the function, including the code that gets executed when the function is called. Functions are defined in ".cpp" files.

- It includes the function's name, return type, parameters, and the body of the function enclosed in curly braces ({}).

```
bool isButtonPressed(PinName pin) {
    DigitalIn button (pin);
    button.mode(PullDown); // Set pull-down mode

    if (!button) { // Check if button is pressed
        return true;
    } else {
        return false;
    }
}
```

Variable Scope Summary

Scope	Declared Where?	Visibility	Lifetime	Embedded Best Practice
Local	Inside a function	Only that function	Function call duration	✅ Preferred – safe, low coupling
Static Local	Inside a function, with *static*	Only that function	Entire program	✅ Good – preserves state without globals
File-static(global)	At file level, with *static*	Only that *.c* file	Entire program	✅ Acceptable – module-private data
Global (external)	At file level, **without** *static*	Entire program (all files)	Entire program	❌ Avoid – increases coupling, hard to debug

- Use **local** variables whenever possible: they are fast and self-contained.
- Use ***static* local** to retain a value between calls (e.g., debounce counter).
- Use **file-*static*** variables for data shared only within one module (e.g., internal state of a driver).
- Avoid **true global variables** – they create hidden dependencies. If unavoidable, document thoroughly.

Variable Scope Summary (Extra Material for Reading)

Scope	Lifetime	What Happens
Local	Function call duration	Created when function starts, destroyed when function returns. Memory is usually on the stack.
Static Local	Entire program	Created once before *main()*, persists across multiple function calls. Value is preserved between calls.
Global/File-static	Entire program	Created before *main()*, destroyed after *main()* exits. Exists as long as the program runs.

- In simple terms:
- **Local variable:** a whiteboard inside a meeting room. Erased when the meeting ends.
- **Static local:** a locked drawer in the same room. Contents remain even after the meeting ends. Next meeting finds the same notes.
- **Global:** a notice board in the building lobby. Everyone sees it, and it stays up forever.

Local vs *static* Local Variables (Extra Material for Reading)

Aspect	Local (automatic)	Static Local
Declaration	*int counter;*	*static int counter;*
Creation	Each time function is called.	Once, before *main()*
Initialisation	Each call (value is indeterminate if not initialised)	Once (initialised to 0 by default, or to explicit value)
Value persistence	Lost when function returns	Preserved between function calls
Use Cases	Temporary calculations	Counters, state that must survive across calls (e.g., debounce filter)

```c
void demo(void) {
    int local = 0;           // Reset to 0 on EVERY call
    static int static_local = 0; // Initialised once only

    local++;
    static_local++;

    printf("local: %d, static_local: %d\n", local, static_local);
}
```

```
local: 1, static_local: 1
local: 1, static_local: 2
local: 1, static_local: 3
```

Benefits of Modular Programming (This used to be two slides)

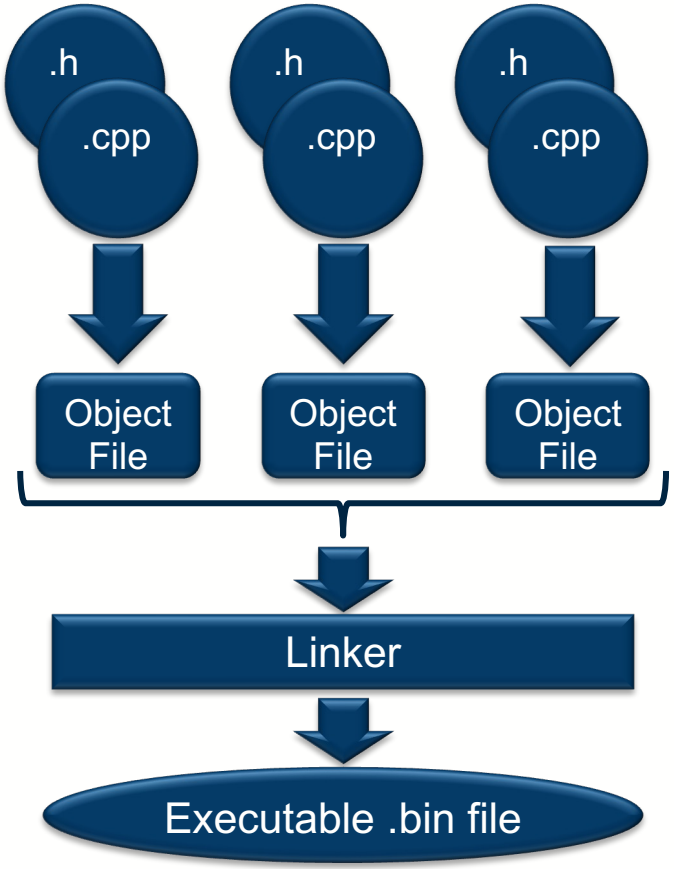
- **Code Reusability and Manageability:** Having a program broken into smaller sub-programs allows for easier management. The separate modules are easier to test, implement or design. These individual modules can then be used to develop the whole program.
- **Easier Error Detection and Debugging:** Modular Programming minimises the risks of ending up with programming errors and also makes it easier to spot errors, if any. This is because the errors can be narrowed down to a specific module, function, or a sub-program.
- **Improves Code Readability:** Modular Programming helps develop programs that are much easier to read since they can be enabled as user-defined functions. A program that carries multiple functions is easier to follow.
- **Easier to Collaborate on Software Development:** With Modular Programming, programmers can collaborate and work on the same application. Designing a program also becomes simpler because small teams can focus on individual parts of the same code.
- **Easier to Scale Up or Improved Scalability:** Allows the program to grow in complexity while maintaining organisation and manageability.

Modular Programming: How?

- Modular coding uses header files to join multiple files together.
- In general a program will use the *main.cpp* file to call and use functions, which are defined in feature specific .cpp files.
- Each .cpp definition file should have an associated declaration file, called the 'header file', e.g., *buzzer.h*, *buzzer.c*).
- Header files have a .h extension and typically include declarations only, for example compiler directives, variable declarations and function prototypes.
- A number of header files also exist from within the Mbed OS library. These can be used for more advanced data manipulation or other functions.

Modular Programming: How are programs pre-processed, compiled and linked?

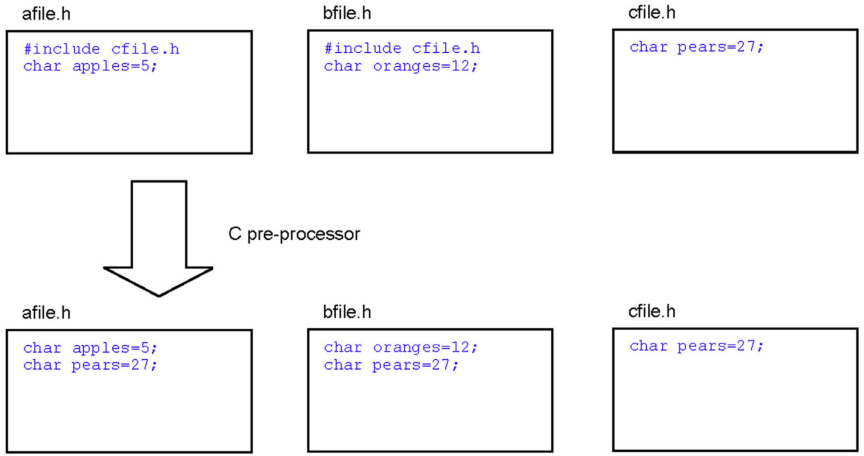
- A pre-processor looks at a particular source (.cpp) file and implements any pre-processor directives and associated header (.h) files.
  - Pre-processor directives are denoted with a '#' symbol.
- The compiler then generates an object file for the particular source code.
- Object and library files are then linked together to generate an executable binary (.bin) file.



Modular Programming: How?

- The #include directive is used to tell the pre-processor to include any code or statements contained within an external header file.

- It is important to note that *#include* acts as a cut and paste feature. If you include "*afile.h*" and "*bfile.h*" where both files also include "*cfile.h*" You will have two copies of the contents of "*cfile.h*" (variable *pears* will be defined twice). The compiler will therefore highlight an error, as multiple declarations of the same variables or function prototypes are not allowed.

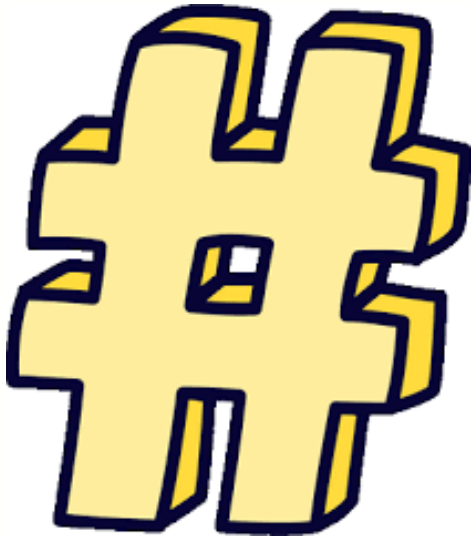


Modular Programming: How?

- **Header Guard Techniques: *#ifndef* vs *#pragma once***
- Header (of Include) Guards: Prevent a header file from being included multiple times in the same translation unit (avoid redefinition errors).

```c
#ifndef LED_MODULE_H
#define LED_MODULE_H

void led_init(void);
void led_on(void);
void led_off(void);

#endif // LED_MODULE_H
```

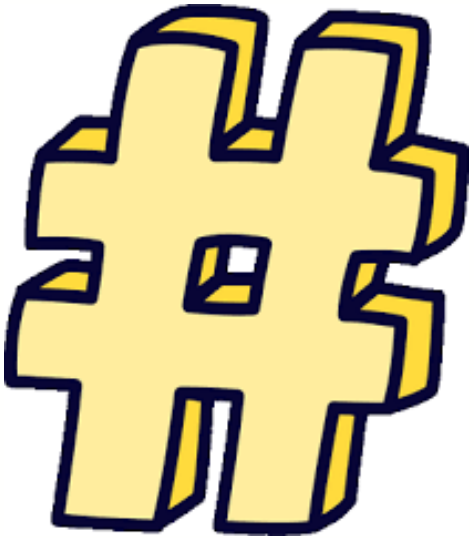
```c
#pragma once

void led_init(void);
void led_on(void);
void led_off(void);
```



- ***#ifndef*** Preprocessor checks if *LED_MODULE_H* is not defined; includes code only once. It works on every C/C++ compiler – standard ().
- ***#pragma once*** Compiler tracks which files have been included and skips repeats automatically. It is supported by GCC, Clang, ARMCC, IAR, MSVC, i.e. most modern embedded compilers – but it is non-standard.

Modular Programming: How? (Extra Material for Reading)

Feature	*#ifndef*	*#pragma once*
Standardised	Yes	No
Portability	All compilers	Most modern compilers
Typo risk	Yes (macro name mismatch)	NA – No name needed
Compile time	Slightly slower (file re-opens)	Faster (compiler caches)



Modular Programming: How?

- Since multiple declarations of a variable or function prototype are not allowed and that it is important for header files to include all the variables and features that are required for the linker to build the project successfully. It is therefore necessary to ensure that all variables and function prototypes are preprocessed ***once***.

```
#ifndef BUTTON_H
#define BUTTON_H

#include "mbed.h"

bool isButtonPressed(PinName pin);

#endif
```

- We can use the *#ifndef* pre-processor directive to ensure that header code is only ever included once by the linker. This is called a header or include guard.

- *#ifndef* means '**if not defined**'.

Recap: What Have We Learned Today?

- Modular programming is a process of breaking down code into individual parts
- Modular programming has several benefits:
  - Code reusability
  - Scalability
  - Readability
  - Simplified debugging, etc.
- We can achieve modular programming by separating different functionalities into different header files and corresponding .c or .cpp files

Summary

- Why Modular Programming?
- What is Modular Programming?
- How to build a modular program?